A Two-Stage Wake-Word System for Keyword Spotting on Arduino Nano 33 BLE Sense

Cappellaro Nicola
nicola.cappellaro@studenti.unitn.it
University of Trento
Povo, Trento, Italy

Zannoni Riccardo
riccardo.zannoni@studenti.unitn.it
University of Trento
Povo, Trento, Italy

ABSTRACT

We present *HeyNano*, a low-power, always-on keyword spotting (KWS) system deployed on an Arduino Nano 33 BLE Sense Rev 2 microcontroller. The system continuously listens for the custom wake word *"heynano"* and, upon detection, enters an activation window in which it recognises the commands *"on"* and *"off"* to toggle the on-board LED. A compact 1D convolutional neural network (2,029 parameters, ≈8 KB float32 or ≈2 KB int8) is trained on a 5-class dataset of roughly 3,256 one-second audio clips and achieves **83.46%** top-1 accuracy on the held-out test set. The model is converted to a full int8 TensorFlow Lite (TFLite) flatbuffer, embedded as a C array, and executed in-place with TFLite Micro on the Cortex-M4 core of the nRF52840, consuming no external memory and enabling real-time inference at 16 kHz with a latency well below the 1-second recognition window.

1 MOTIVATION

The proliferation of battery-powered IoT edge devices has created strong demand for *always-on* voice interfaces that operate without cloud connectivity and within tight energy and memory budgets. Traditional approaches either rely on a dedicated, always-on DSP for wake-word detection (high cost, proprietary) or offload inference to the cloud (high latency, no offline capability, privacy concerns).

Keyword spotting on a raw microcontroller strikes the ideal balance: the device sleeps at very low power, the PDM microphone and a few kilobytes of SRAM are enough to detect a specific spoken command, and no sensitive audio ever leaves the chip. The specific problem addressed here is:

- **On-device wake-word detection with custom vocabulary.** Consumer assistants (Alexa, Google, Siri) use proprietary wake words and require network connectivity. We want the same capability on a $20 microcontroller with an open-source toolchain.
- **Two-stage command recognition.** After the wake word "heynano" is detected the device must distinguish the commands "on" and "off" within a 3-second window — mimicking the interaction model of commercial smart-home devices but running entirely at the edge.
- **Extreme resource constraints.** The Arduino Nano 33 BLE Sense Rev 2 mounts an nRF52840 (Cortex-M4 @ 64 MHz, 1 MB Flash, 256 KB SRAM). Both the model weights and the inference runtime must fit comfortably in these limits.

2 LIMITATIONS OF THE STATE OF THE ART

Existing approaches each carry significant limitations on ultra-constrained MCUs. **Cloud-based assistants** (Alexa, Google, Siri) require persistent connectivity, incur high latency, raise privacy concerns, and do not support custom vocabularies. **Dedicated neural ASICs** (e.g. Syntiant NDP [9], Ambiq Apollo [10]) offer low-power always-on detection but rely on proprietary SDKs and tooling, and eliminate the general-purpose compute needed for a second recognition stage. **TFLite Micro reference designs** [1] fix the vocabulary to "yes"/"no", hard-wire a C-only feature pipeline that is non-trivial to replicate in Python for training, and provide no two-stage interaction model, which increases false-positive rates. Our work addresses all three limitations with a fully open-source pipeline, a custom in-house vocabulary, and an explicit two-stage state machine.

3 KEY INSIGHTS

The two core insights that make this system effective are:

1. Strict feature-extraction parity between training and inference. On-device accuracy depends critically on running the *same* DSP pipeline during inference as during training. The TFLite Micro `microfrontend` implements a specific sequence of steps (noise reduction, PCAN gain control, mel filterbank, log scaling, int8 quantisation) whose exact numerical behaviour is non-trivial to replicate in Python. We reverse-engineered each step — including the noise-reduction smoothing constants ($\alpha_{\text{even}} = 0.025$, $\alpha_{\text{odd}} = 0.060$), PCAN parameters (strength = 0.95, offset = 80), log-scale shift (divide by 2^6) and output scaling (multiply by 2^{12}), and the int8 re-quantisation formula $v = \lfloor(\text{raw} \times 256 + 333)/666\rfloor - 128$ — and implemented them exactly in NumPy/TensorFlow so that training features are byte-identical to on-device features.

For this reason, despite MFCC features achieving higher accuracy in our ablation study (up to 88.46%, Table 5), we ultimately selected the LFBE pipeline: MFCC adds a Discrete Cosine Transform (DCT) stage whose numerical behaviour in the TFLite Micro `microfrontend` proved difficult to replicate exactly in Python, introducing a feature mismatch that degraded on-device accuracy. LFBE, by contrast, stops before the DCT and maps directly onto the `microfrontend` output, making byte-exact replication tractable.

2. Two-stage state machine with adaptive averaging windows. A single-stage recogniser that is always looking for all keywords is prone to false positives on "on" and "off", which are short, common phonemes. By introducing a state machine (IDLE → ACTIVATED → IDLE), the device only evaluates "on"/"off" after the low-confusion wake word "heynano" has been confirmed. Furthermore, different temporal averaging windows are used in each stage: a 1000 ms window in IDLE for robust wake-word detection and a shorter 600 ms window in ACTIVATED for faster command response. A 1500 ms suppression interval prevents double-counting of the same utterance.

4 MAIN ARTIFACTS

The project produced four main artifacts: a custom audio dataset, a Python feature-extraction and training notebook, an int8 TFLite model, and an Arduino sketch.

4.1 Dataset

Audio data were collected and managed with Edge Impulse [2]. The dataset comprises five classes:

Table 1: Dataset composition per class.

Class	Source	N° of samples
heynano	recorded in-house	606
on	in-house / Google Speech	608
off	in-house / Google Speech	603
noise	Edge Impulse dataset	719
unknown	Edge Impulse dataset	720
Total		**3,256**

Each sample is a 1-second mono WAV file sampled at 16 kHz. The total recording time is approximately 55 minutes. The "heynano" samples were recorded by the authors in diverse conditions (different speakers, microphones, and ambient noise levels) to maximise generalisation. After collection the full corpus was split by Edge Impulse into: **Training: 2,615 samples** and **Test: 641 samples** (roughly 80%/20%).

4.2 Feature Extraction Pipeline

Each 1-second audio clip is transformed into a 49×32 matrix of Log Filter-Bank Energies (LFBE), mirroring the TFLite Micro `microfrontend` exactly (see Section 3). Table 2 summarises the DSP parameters.

Table 2: DSP / feature-extraction parameters.

Parameter	Value	Notes
Sample rate	16,000 Hz	mono
Frame length	25 ms	400 samples
Frame stride	20 ms	320 samples
FFT size	512	next power of 2 ≥ 400
Mel channels	32	$f_{\min} = 125$ Hz $f_{\max} = 7{,}500$ Hz
Slices per window	49	covers exactly 1 s
Output type	int8	quantised
Feature tensor size	$49 \times 32 = 1{,}568$	flattened for model input

The pipeline mirrors the TFLite Micro `microfrontend` exactly (see Section 3): noise reduction with exponential smoothing, PCAN gain control, a 32-channel HTK-scale mel filterbank (125–7,500 Hz), log-domain compression, and int8 re-quantisation to $[-128, 127]$.

4.3 Neural Network Model

We trained a lightweight 1D convolutional network using TensorFlow/Keras. The model is deliberately small to fit within the MCU Flash budget and to execute quickly on a single Cortex-M4 core without hardware FPU acceleration for int8 ops. The architecture is shown in Table 3.

Training used the Adam optimiser with categorical cross-entropy loss for 100 epochs (batch size 32, 80%/20% train/-validation split from the training set). The final architecture (experiment #6 in Table 6) was selected after an ablation study across 14 configurations per feature regime (MFCC and LFBE), spanning MLP baselines, compact CNNs, and quantisation-aware variants; full results are reported in Appendix A.

Comparison with DNN baselines. As part of the architecture search, we evaluated a fully-connected DNN baseline following the procedure adopted by the Syntiant NDP101 speaker-verification pipeline [12]. Three variants of a three-layer MLP (Dense 32-16-8) were tested: a standard baseline, a Quantization-Aware Training (QAT) version, and a Post-Training Quantization (PTQ) version (experiments #15–17 in Table 6). All three variants share the same 50,917-parameter footprint, which is **more than 25 times larger** than our

Table 3: Model architecture (2,029 trainable parameters).

Layer	Output shape	Params
Input (flat)	(1,568)	0
GaussianNoise(σ=0.1)	(1,568)	0
Reshape	(49, 32)	0
Conv1D(8, k=5, ReLU)	(49, 8)	1,288
MaxPooling1D(2)	(25, 8)	0
Dropout(0.25)	(25, 8)	0
Conv1D(16, k=5, ReLU)	(25, 16)	656
MaxPooling1D(2)	(13, 16)	0
Dropout(0.25)	(13, 16)	0
GlobalAveragePooling1D	(16)	0
Dense(5, Softmax)	(5)	85
Total		**2,029**

selected 1D CNN (2,029 parameters), yet achieve only 71.14%–73.48% accuracy — **below** the 83.46% of the deployed convolutional model. This confirms that, for this task and dataset size, local feature reuse through convolution is strictly more parameter-efficient than dense connectivity: the MLP must learn spatial correlations from scratch across the full 49×32 input, whereas the convolutional filters exploit the temporal structure of the mel spectrogram at a fraction of the cost. QAT slightly recovers accuracy over the float32 baseline (73.48% vs. 71.14%), suggesting that the quantisation noise acts as a mild regulariser at this scale, but neither variant closes the gap with the CNN.

4.4 Quantisation and Model Conversion

After training in float32, the model is converted to a **full int8** TFLite flatbuffer via post-training quantisation (PTQ), calibrated on 300 representative training samples with `tf.lite.Optimize.DEFAULT` and `TFLITE_BUILTINS_INT8` ops, setting both input and output types to `tf.int8`. The resulting `best.tflite` file is converted to a C array with `xxd -i`, annotated with `alignas(16) const`, and linked under the symbols `g_model_data` and `g_model_data_len`.

4.5 Arduino Deployment Sketch

The Arduino IDE sketch `heynano_voice_control.ino` implements the full real-time pipeline on the nRF52840: PDM capture → microfrontend → TFLite Micro inference → score averaging → two-stage state machine → LED/serial output.

Audio capture. The PDM library triggers a DMA callback (`CaptureSamples()`) approximately every 16 ms, writing 256 int16 samples into a circular ring buffer 16 times the DMA transfer size. The main loop polls a volatile timestamp to gate processing on new audio.

TFLite Micro setup. The operator resolver registers only the ops used by the model, minimising Flash usage. A static tensor arena is allocated on the stack; its exact size is printed to Serial at startup.

Score averaging and suppression. Raw int8 output scores are accumulated into a circular queue of `InferenceResult` structs timestamped in milliseconds. At each step, scores within the active window (1000 ms in IDLE, 600 ms in ACTIVATED) are averaged per class. A detection fires only when (a) the averaged top-class score exceeds the threshold $T = 200$ (out of 255) and (b) more than $N_{\text{min}} = 3$ inferences are available in the window. A 1500 ms suppression interval prevents the same utterance from triggering twice.

Two-stage state machine.

- **IDLE:** Only the "heynano" class is considered for a detection event. On detection, the green LED flashes for 500 ms, the score queue is flushed, and the machine transitions to ACTIVATED.
- **ACTIVATED:** Only "on" and "off" are acted upon. "on" turns the built-in LED on; "off" turns it off. Both commands return immediately to IDLE. If neither command is detected within 3 000 ms, the machine silently returns to IDLE (timeout path), avoiding an indefinitely open activation window.

5 KEY RESULTS AND CONTRIBUTIONS

Empirical Results

Table 4: Summary of key metrics.

Metric	Value
Test-set accuracy (float32)	83.46%
Model parameters	2,029
Float32 model size	7.93 KB
Int8 TFLite model size	≈2 KB
Tensor arena	64 KB ≪ 256 KB SRAM
Inference throughput	∼20 inferences/s (1 per 20 ms stride)
Detection latency (IDLE)	≤1,000 ms averaging window
Detection latency (ACTIVATED)	≤600 ms averaging window
Hardware platform	Arduino Nano 33 BLE Sense Rev 2
MCU	nRF52840 (Cortex-M4 @ 64 MHz)

Technical Contributions

- **Custom wake-word dataset with in-house recordings.** We recorded and published a "heynano" class that does not exist in any existing open-source speech corpus, together with on/off extensions, making the dataset immediately reusable for future work on this wake word.
- **Byte-exact Python reimplementation of the TFLite Micro `microfrontend`.** We identified and reproduced every numerical step of the on-device DSP pipeline in NumPy/TensorFlow, including noise reduction, PCAN gain control, log scaling, and int8 re-quantisation. This closes the *train/inference gap* that commonly degrades accuracy when deploying audio ML models on microcontrollers.
- **Ultra-compact 1D CNN architecture.** The 2,029-parameter model demonstrates that acceptable KWS accuracy (>80%) is achievable with a network two orders of magnitude smaller than DS-CNN or MobileNet variants [11], making the approach viable for the most resource-constrained MCU class.
- **Two-stage state machine with adaptive averaging.** Separating wake-word detection (wide window, 1000 ms) from command recognition (narrow window, 600 ms) improves both the reliability of wake-word detection and the responsiveness of command execution, while the 3-second timeout prevents the device from remaining in a permanently activated state.
- **Fully open-source, end-to-end pipeline.** Dataset collection (Edge Impulse), feature extraction and training (Python / TensorFlow), quantisation (TFLite PTQ), model embedding (`xxd`), and firmware (Arduino IDE / TFLite Micro) are all based on freely available tools and are described in sufficient detail to be reproduced from scratch.

Limitations and Future Work

The current system has several known limitations. First, 83.46% accuracy, while reasonable for a 2K-parameter model, means roughly 1 in 5 test samples is misclassified; accuracy could be improved with data augmentation (speed perturbation, room impulse response convolution) or a slightly larger model. Second, the dataset was recorded primarily by a small number of speakers; broader speaker variability would improve generalisation. Third, the system does not yet exploit the nRF52840's built-in low-power modes (Bluetooth sleep, CPU idle); integrating proper power management would allow true always-on operation at sub-mW average power. Future work will explore on-device continual learning to personalise the wake word to a specific user's voice.

A ARCHITECTURE SEARCH: FULL ABLATION RESULTS

The architecture search was conducted independently under two feature regimes: Mel-Frequency Cepstral Coefficients (MFCC) and Log Filter-Bank Energies (LFBE, the pipeline described in Section 4.2). Each regime tested 14 and 17 configurations, respectively spanning compact CNNs, ultra-lightweight CNNs with GlobalAveragePooling, and MLP baselines. MFCC features consistently yielded higher accuracy across equivalent architectures; **Table 5 and Table 6** in the appendix illustrate the results obtained across the evaluated configurations.

REFERENCES

[1] P. Warden and D. Situnayake, *TinyML: Machine Learning with TensorFlow Lite on Arduino and Ultra-Low-Power Microcontrollers*. O'Reilly Media, 2020.

[2] J. Plunkett et al., "Edge Impulse: An MLOps Platform for Tiny Machine Learning," *Proceedings of Machine Learning and Systems*, 2023. https://docs.edgeimpulse.com/tutorials/end-to-end/keyword-spotting

[3] Edge Impulse, "Keyword Spotting Audio Dataset," *Edge Impulse Documentation*, 2023. https://docs.edgeimpulse.com/datasets/audio/keyword-spotting

[4] GeeksforGeeks, "Mel Frequency Cepstral Coefficients (MFCC) for Speech Recognition," *GeeksforGeeks NLP Series*, 2024. https://www.geeksforgeeks.org/nlp/mel-frequency-cepstral-coefficients-mfcc-for-speech-recognition/

[5] TensorFlow Authors, "TensorFlow Lite Micro Arduino Examples," *GitHub Repository*, 2024. https://github.com/tensorflow/tflite-micro-arduino-examples

[6] P. Warden, "Speech Commands: A Dataset for Limited-Vocabulary Speech Recognition," *arXiv:1804.03209*, 2018.

[7] Y. Zhang et al., "Hello Edge: Keyword Spotting on Microcontrollers," *arXiv:1711.07128*, 2017.

[8] R. David et al., "TensorFlow Lite Micro: Embedded Machine Learning on TinyML Systems," *Proceedings of MLSys*, 2021.

[9] Syntiant Corp., "NDP120 Neural Decision Processor," *Product Brief*, 2022. https://www.syntiant.com/ndp120

[10] Ambiq Micro, "Apollo4 SoC Family: Ultra-Low Power Always-On Voice Processing," *Product Announcement*, 2020. https://ambiq.com/news/apollo4-soc-family/

[11] A. G. Howard et al., "MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications," *arXiv:1704.04861*, 2017.

[12] TinySystems Lab, "Syntiant NDP101 Speaker Verification Pipeline," *GitHub Repository*, 2023. https://github.com/tinysystems/Syntiant-NDP101-Speaker-Verification

Table 5: Ablation with MFCC (50 × 13) features (14 configurations).

ID	Experiment	Architecture	Params	Accuracy
1	Baseline CNN (3 layers, growing filters)	3 Conv (8-16-32)	3,413	88.30%
2	Compact CNN (3 layers, reduced filters)	3 Conv (4-8-16)	1,229	83.46%
3	Simplified CNN (2 layers)	2 Conv (8-16)	1,765	86.43%
4	**CNN + GlobalAveragePooling (kernel 5)**	**2 Conv (8-16) + GAP**	**1,269**	**88.46%**
5	Ultra-lightweight CNN (kernel 2 + GAP)	2 Conv (8-16) + GAP	573	79.88%
6	CNN with light dropout	2 Conv (8-16) + Dropout 0.1	1,765	85.80%
7	CNN + GAP with regularisation	2 Conv (8-16) + GAP + Dropout	805	84.71%
8	Minimal CNN (reduced filters)	2 Conv (4-8)	789	82.06%
9	Minimal CNN with dropout	2 Conv (4-8) + Dropout 0.1	789	83.78%
10	Minimal CNN without regularisation	2 Conv (4-8)	789	81.59%
11	Ultra-compact CNN with GAP	2 Conv (4-8) + GAP	309	76.29%
12	MLP baseline	Dense (32-16-8)	21,564	80.97%
13	MLP + Quantization Aware Training	Dense (32-16-8)	21,564	81.59%
14	MLP + Post-Training Quantization	Dense (32-16-8)	21,564	81.12%

Table 6: Ablation with LFBE features (17 configurations). Experiment #6 (highlighted) is the model selected for deployment.

ID	Experiment	Architecture	Params	Accuracy
1	Baseline CNN (3 layers, growing filters)	3 Conv (8-16-32)	3,869	82.84%
2	Compact CNN (3 layers, reduced filters)	3 Conv (4-8-16)	1,457	79.10%
3	Simplified CNN (2 layers)	2 Conv (8-16)	2,221	80.97%
4	Simplified CNN with dropout	2 Conv (8-16) + Dropout 0.1	2,221	83.15%
5	Simplified CNN without dropout	2 Conv (8-16)	2,221	79.25%
6	**CNN + GAP (kernel 5)**	**2 Conv (8-16) + GAP**	**2,029**	**83.46%**
7	Ultra-lightweight CNN (kernel 2 + GAP)	2 Conv (8-16) + GAP	877	78.78%
8	CNN + GAP with dropout 0.25	2 Conv (8-16) + GAP + Dropout 0.25	1,261	81.44%
9	CNN + GAP with dropout 0.1	2 Conv (8-16) + GAP + Dropout 0.1	1,261	82.53%
10	CNN + GAP without dropout	2 Conv (8-16) + GAP	1,261	80.66%
11	Minimal CNN (reduced filters)	2 Conv (4-8)	1,017	81.12%
12	Minimal CNN with dropout	2 Conv (4-8) + Dropout 0.1	1,017	76.60%
13	Minimal CNN without dropout	2 Conv (4-8)	1,017	75.66%
14	Minimal CNN + GAP	2 Conv (4-8) + GAP	537	77.69%
15	MLP baseline	Dense (32-16-8)	50,917	71.14%
16	MLP + Quantization Aware Training	Dense (32-16-8)	50,917	73.48%
17	MLP + Post-Training Quantization	Dense (32-16-8)	50,917	71.92%